

Math 189 Homework 2

Daniel Kurland

16 August, 2026

Problem 1

```
library(tidyverse)
alpha <- 0.05
df <- read.csv("Natality, 2007-2024.csv")

clean_df <- df %>%
  filter(
    !is.na(County), nzchar(trimws(County)), County != "Unidentified Counties, CA",
    !is.na(Sex), nzchar(trimws(Sex)),
    !is.na(Births)
  )
county_summary <- clean_df %>%
  group_by(County) %>%
  summarise(
    Total_Births = sum(Births),
    Female_Births = sum(Births[Sex == "Female"]),
    Male_Births = sum(Births[Sex == "Male"]),
    Prop_Female = Female_Births / Total_Births
  )
summary(county_summary$Prop_Female)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.4592  0.4837   0.4876  0.4872  0.4923   0.5040
```

```
contingency_df <- clean_df %>%
  select(County, Sex, Births) %>%
  pivot_wider(names_from = Sex, values_from = Births)

counts_matrix <- as.matrix(contingency_df[, c("Female", "Male")])
rownames(counts_matrix) <- contingency_df$County

counts_table <- xtabs(Births ~ County + Sex, data = clean_df)

## Chi-squared test for homogeneity
## H0: Probability of being a girl is the same in all counties
## H1: Probability of being a girl differs in at least one county
chi2_result <- chisq.test(counts_table)
print(chi2_result)
```

```
##
## Pearson's Chi-squared test
##
## data: counts_table
## X-squared = 34.318, df = 34, p-value = 0.4525
```

```
print(chi2_result$p.value < alpha)
```

```
## [1] FALSE
```

Conclusion: do not reject null hypothesis, no significant evidence that birth sex ratio differs across counties

Problem 2

Part a:

```
chisq.power <- function(k, t, n, B = 2000, alpha = 0.05) {
  reject_vector <- logical(B)
  prob_vector <- c(rep(1 / (2 * k) + t, k), rep(1 / (2 * k) - t, k))

  for (i in 1:B) {
    x <- sample(1:(2 * k), size = n, replace = TRUE, prob = prob_vector)
    counts <- table(factor(x, levels = 1:(2 * k)))
    reject_vector[i] <- chisq.test(counts)$p.value < alpha
  }

  return(mean(reject_vector))
}
```

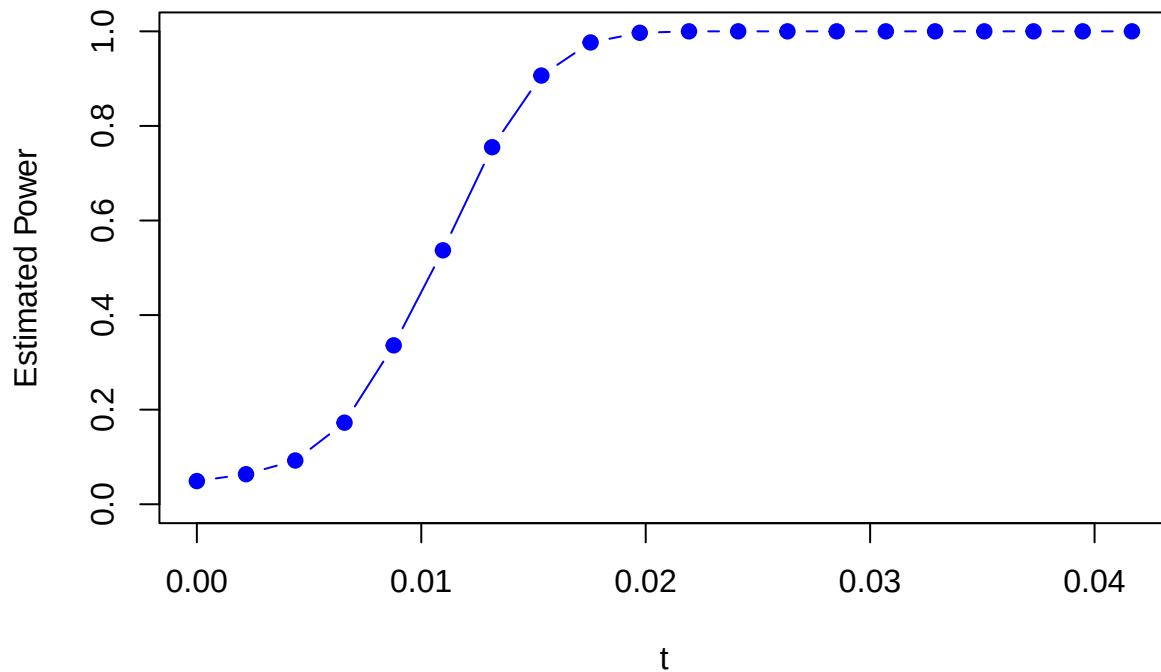
Part b:

```
k_val <- 12
n_val <- 200

t_seq <- seq(0, 1 / (2 * k_val), length.out = 20)
power_vals <- sapply(t_seq, function(t) {
  chisq.power(k = k_val, t = t, n = n_val, B = 2000)
})

plot(
  t_seq, power_vals,
  type = "b",
  pch = 19,
  col = "blue",
  xlab = expression(t),
  ylab = "Estimated Power",
  main = paste("Chi-Squared Test Power Curve (k =", k_val, ", n =", n_val, ")"),
  ylim = c(0, 1)
)
```

Chi-Squared Test Power Curve (k = 12 , n = 200)



Problem 3

```
set.seed(42)
n_sims <- 10000
k_vector <- c(5, 10, 20, 50, 100)
n_vector <- c(2, 4, 5, 6, 8, 10)

results_matrix <- matrix(NA, nrow = length(k_vector), ncol = length(n_vector))
rownames(results_matrix) <- paste0("k=", k_vector)
colnames(results_matrix) <- paste0("n=", n_vector, "k")

expected_count_matrix <- results_matrix

fast_chisq_pval <- function(counts, k, n) {
  expected <- (n / k)
  stat <- sum((counts - expected)^2 / expected)
  pchisq(stat, df = k - 1, lower.tail = FALSE)
}

for (i in seq_along(k_vector)) {
  k <- k_vector[i]
  for (j in seq_along(n_vector)) {
    n <- n_vector[j] * k
    p_values <- numeric(n_sims)

    for (s in 1:n_sims) {
      tab <- tabulate(sample.int(k, size = n, replace = TRUE), nbins = k)
```

```

    p_values[s] <- fast_chisq_pval(tab, k, n)
  }

  results_matrix[i, j] <- mean(p_values < 0.05)
  expected_count_matrix[i, j] <- n / k
}
}

results_matrix

```

```

##          n=2k  n=4k  n=5k  n=6k  n=8k  n=10k
## k=5      0.0386 0.0472 0.0447 0.0446 0.0496 0.0487
## k=10     0.0552 0.0515 0.0497 0.0526 0.0479 0.0464
## k=20     0.0457 0.0499 0.0488 0.0478 0.0487 0.0515
## k=50     0.0514 0.0509 0.0547 0.0474 0.0477 0.0546
## k=100    0.0522 0.0508 0.0514 0.0485 0.0485 0.0490

```

```
expected_count_matrix
```

```

##          n=2k n=4k n=5k n=6k n=8k n=10k
## k=5         2   4   5   6   8   10
## k=10        2   4   5   6   8   10
## k=20        2   4   5   6   8   10
## k=50        2   4   5   6   8   10
## k=100       2   4   5   6   8   10

```

Problem 4

roll die 30 times, 1 million iterations

```

library(parallel)
set.seed(42)
n_sims <- 1000000
n_rolls <- 30
p_null <- rep(1 / 6, 6)

observed_sample <- sample(1:6, size = n_rolls, replace = TRUE)
observed_counts <- tabulate(observed_sample, nbins = 6)

calc_stat <- function(counts, n, p) {
  expected <- n * p
  sum((counts - expected)^2 / expected)
}

t_obs <- calc_stat(observed_counts, n_rolls, p_null)

## 1: chi squared test
time_builtin <- system.time({
  fit_builtin <- chisq.test(observed_counts, p = p_null,
    simulate.p.value = TRUE, B = n_sims)
})

```

```

p_val_builtin <- fit_builtin$p.value

## 2: for loop
time_for_loop <- system.time({
  sim_stats <- numeric(n_sims)
  for (i in 1:n_sims) {
    simulated_counts <- tabulate(sample.int(6, size = n_rolls,
      replace = TRUE), nbins = 6)
    sim_stats[i] <- calc_stat(simulated_counts, n_rolls, p_null)
  }
  p_val_loop <- mean(sim_stats >= t_obs)
})

## 3: parallel computing
time_parallel <- system.time({
  num_cores <- max(1, detectCores() - 1)
  cl <- makeCluster(num_cores)

  clusterSetRNGStream(cl, 123)
  clusterExport(cl, varlist = c("n_rolls", "p_null", "calc_stat"), envir = environment())

  null_stats_parallel <- unlist(parLapply(cl, 1:n_sims, function(i) {
    sim_counts <- tabulate(sample.int(6, size = n_rolls, replace = TRUE), nbins = 6)
    calc_stat(sim_counts, n_rolls, p_null)
  }))

  stopCluster(cl)
  p_val_parallel <- mean(null_stats_parallel >= t_obs)
})

## Results Comparison
results <- data.frame(
  Method = c("chisq.test", "For Loop", "Parallel Computing"),
  Elapsed_Time_Sec = c(time_builtin["elapsed"],
    time_for_loop["elapsed"], time_parallel["elapsed"]),
  p_value = c(p_val_builtin, p_val_loop, p_val_parallel)
)

print(results)

```

```

##           Method Elapsed_Time_Sec  p_value
## 1      chisq.test           87.91 0.2492848
## 2        For Loop           12.23 0.2431410
## 3 Parallel Computing           7.39 0.2428350

```